KEYLOOP TECHNICAL ASSESSMENT · SCENARIO A · SYSTEM DESIGN

Unified Service Scheduler

Confirm a workshop booking only when **a qualified technician and a bay are both free for the whole job**, and stay correct when many customers book the same slot at once.

v1 · 2026-11-03 C# .NET 10 · Apache Kafka · PostgreSQL 16

> **Decision:** one writer per dealership. Requests queue on a Kafka partition keyed by dealership; its owner decides in memory and commits in batches. Postgres exclusion constraints stay as a backstop.

0	10 ms	7,919/s	300 / 300
double bookings in 153 benchmark runs, all tests and the demo stack	worst answer, 200 customers on one slot (database-only: never finished)	answers within 500 ms at peak load, on ≤ 16 DB connections	bookings answered while a worker was killed mid-traffic

1 Requirements

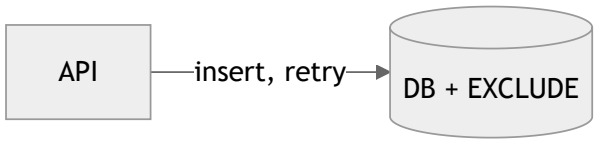
R1 · Book
Vehicle + service + dealership + time → `POST /appointments`

R2 · Check
Qualified technician *and* bay free for the whole slot, even under concurrency

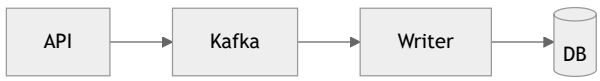
R3 · Record
Persist the appointment with customer, vehicle, technician and bay

2 Options: where does the rule live?

O1 · in the database



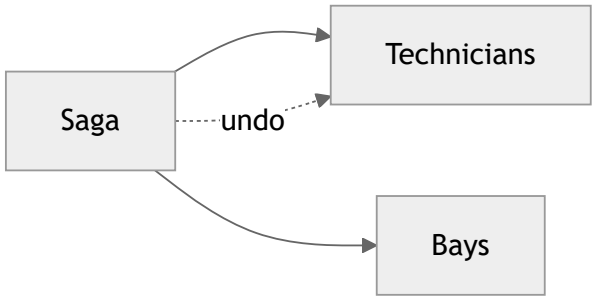
O2 · in one writer per dealership ✓



O3 · in an event stream



O4 · split across services



Scored on paper (safety, depth, scale, risk, testability, footprint)



O1 led on paper, so the two unknowns were measured before deciding.

3 Measured before choosing

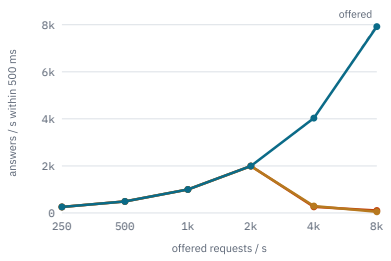
153 runs, 3 trials per point, same schema and request streams, overlap check after every run (`bench/BookingConcurrency`).

O1 constraint + retry · O1 + per-day lock · O2 single writer

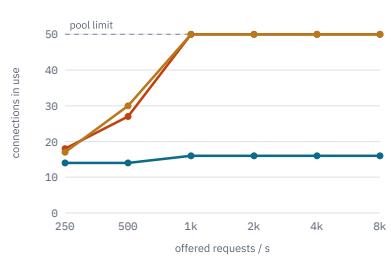
Burst: N customers, one slot



Throughput, normal day

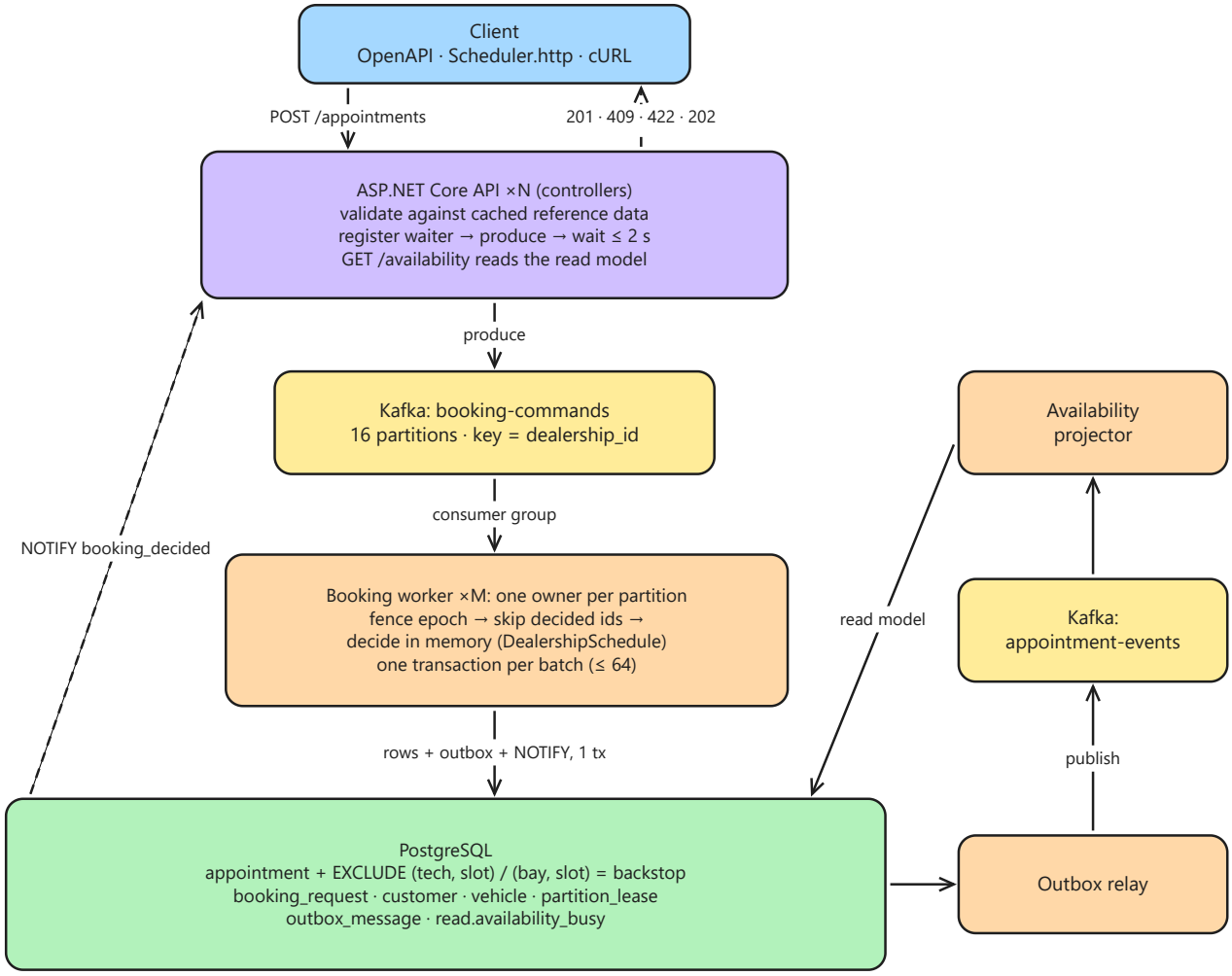


Database connections



- **All correct:** zero overlaps everywhere.
- **O1 deadlocks under bursts**; with a lock it works but waits grow (34 → 300 ms) and it plateaus at ≈ 2,000/s on all 50 connections.
- **O2:** every burst in ≈ 10 ms, 7,919/s on ≤ 16 connections; costs ≈ 5–10 ms per request.

4 Architecture



API, workers, relay and projector export traces, metrics and logs over OTLP (the Aspire dashboard in docker compose). One trace runs from POST /appointments through the Kafka publish to the worker's booking decision.

Command log, scored



A Postgres queue would bring back O1's commit-per-request ceiling.

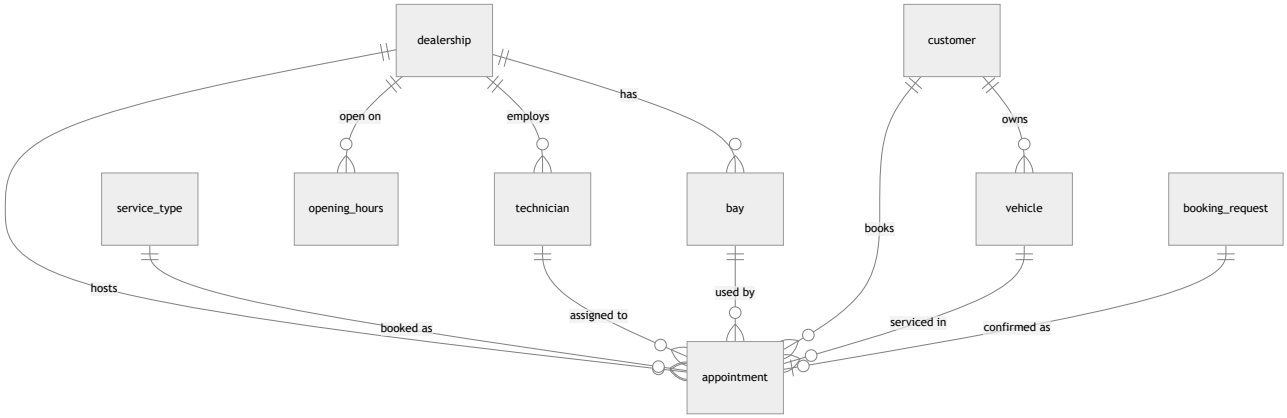
Reply

Worker NOTIFYs on commit → API answers **201 / 409 / 422** within 2 s, else **202** + poll link.

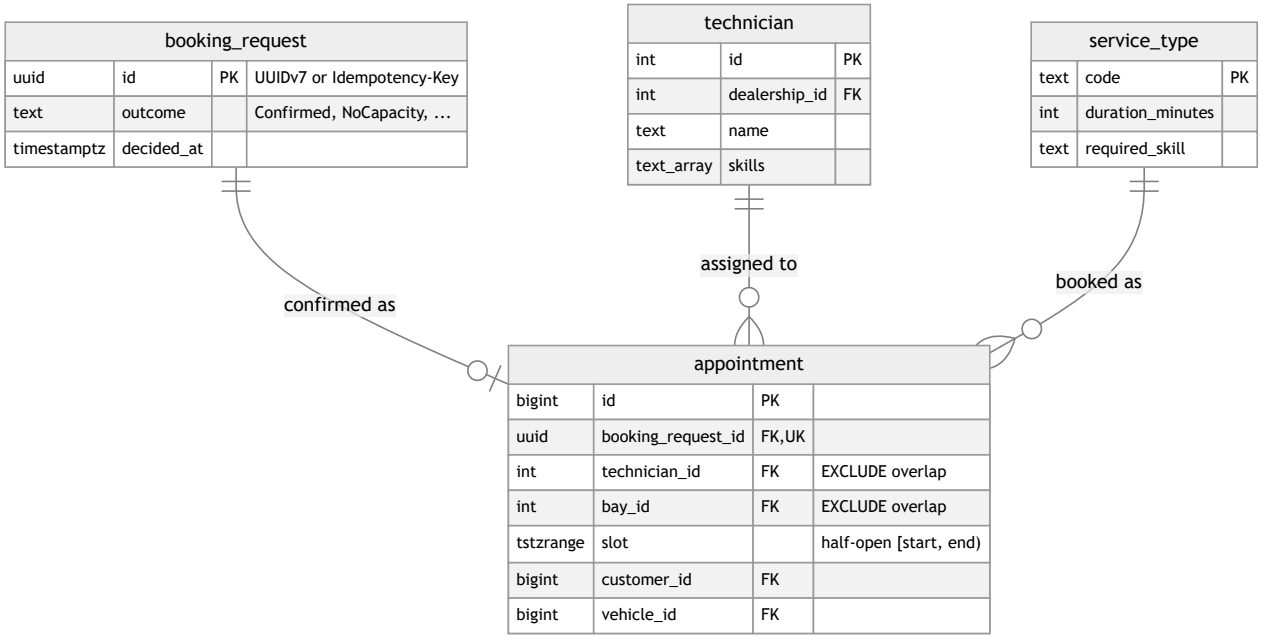
The API never writes to the database on the request path.

5 Data model

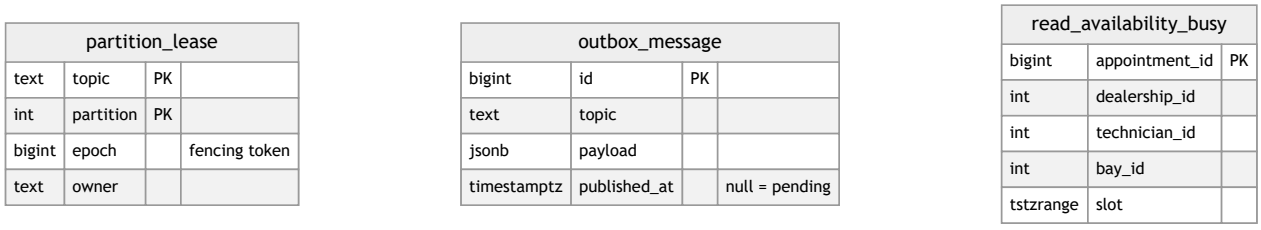
Booking core: tables and relationships



Key tables



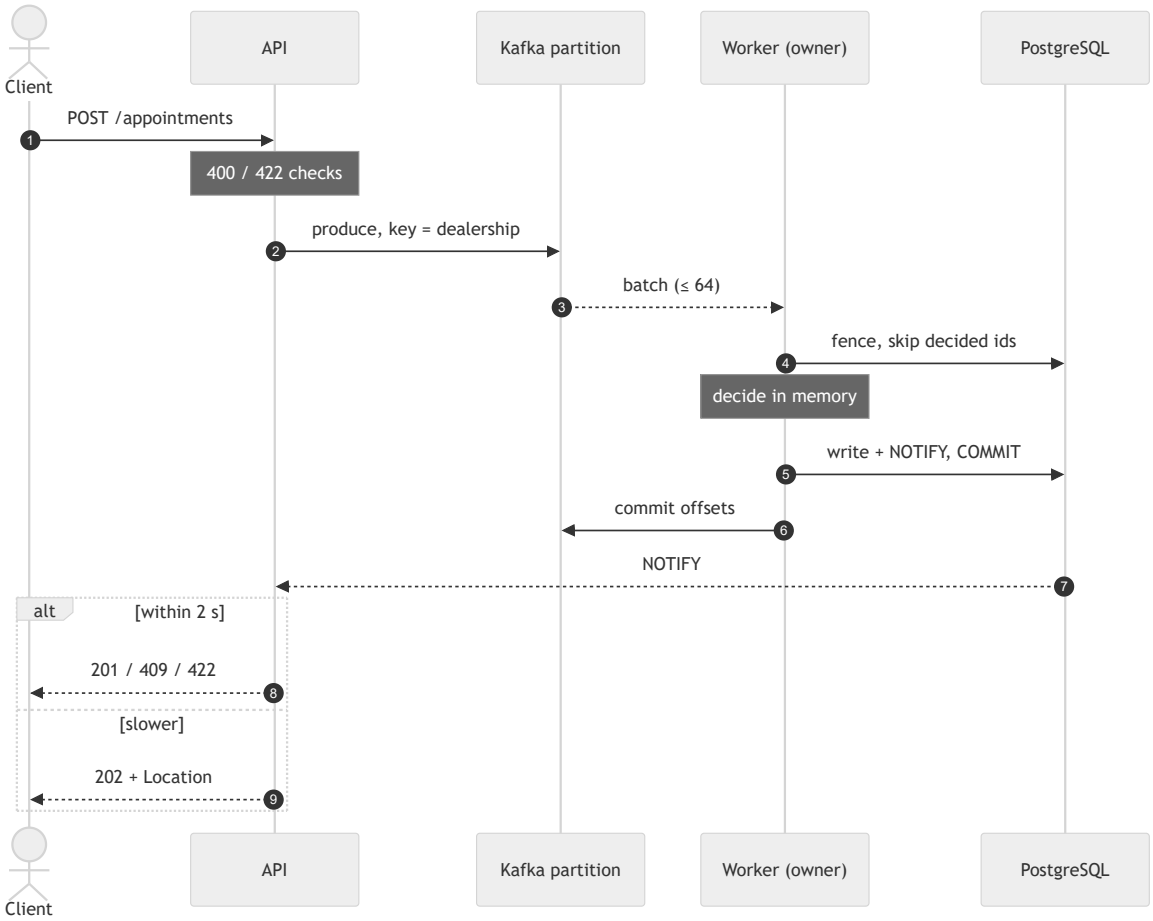
Infrastructure tables



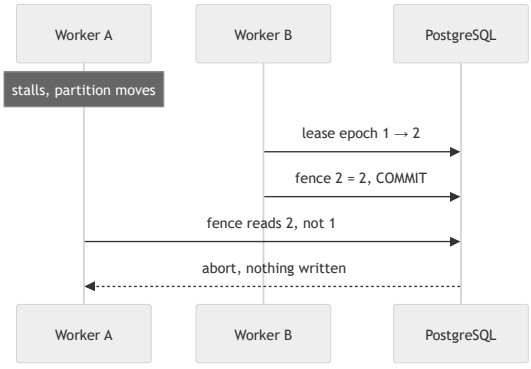
Two GiST exclusion constraints on `appointment` make an overlapping technician or bay impossible in storage. `partition_lease` fences stale workers, `outbox_message` feeds events, `read.availability_busy` is the projected read model.

6 Key flows

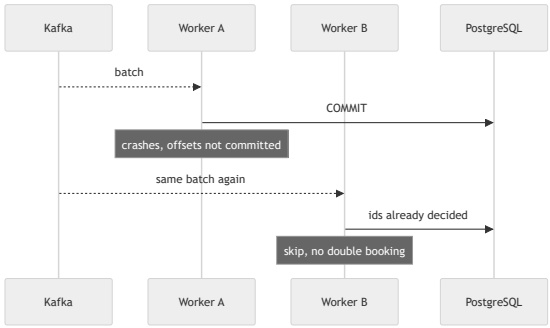
Booking request



Worker loses its partition



Crash before offset commit



7 Proof

Guarantee	Mechanism	Evidence
No overlap	Single writer + EXCLUDE	50 on one slot → 6 × 201, 44 × 409, 0 overlaps
Qualified only	Skill filter first	Unit + integration tests
No stale writer	Fencing epoch	Fencing test
No double effect	Offsets after commit, dedupe	Replay test
Safe retries	Idempotency-Key	Idempotency test
Survives worker loss	Consumer-group failover	Compose: 300 / 300 answered, p99 1.15 s

20 unit tests and 12 integration tests on real Postgres and Kafka (Testcontainers). Traces span API → Kafka → worker (metrics: `decision_latency`, `consumer_lag`, `fence_rejections`, `backstop_violations`).

8 Limits and next steps

- Single broker and database in Compose; production needs replication.
- Read model shares the write database; a replica would isolate it.
- Partition count (16) is fixed; changing it needs a remapping plan.
- Not built: hold-then-confirm, cancellation, authentication.

9 GenAI

- AI implemented; decisions and weights stayed mine (`docs/DECISIONS.md`).
- Its claim "losers get `23P01`" was wrong (deadlocks), which led to the benchmark.
- Tests and the running stack caught its bugs: seed SQL, validation keys, cold start (`docs/AI-LOG.md`).

Details: README · docs/DESIGN.md · docs/DECISIONS.md · docs/benchmarks/ · generated by docs/system-design/build.py